# Documentación técnica

Arquitectura, componentes, despliegue, casos de uso, seguridad e instalación

## Programación 4 · Sistema Web para Gimnasio · Grupo 2

Integrantes: María Ramírez Elizondo; Roberto Chavarría García; Antonio Scala González; Cristopher Madrigal Masis.

Edición de trabajo: 06/11/2026 · Estado: validación técnica y entrega final en preparación.

| Alcance de esta versión | Evidencia técnica |
| --- | --- |
| React + Vite + Java Spring Boot + JPA + MariaDB + SOAP | 9 pruebas automatizadas Java aprobadas; Vite build y lint aprobados; prueba HTTP integral aprobada sobre gimnasio_entregable6_test. |

No sustituye la evidencia de capturas definitivas de interfaz ni la demostración ante la profesora.

# 1. Alcance y requisitos del sistema

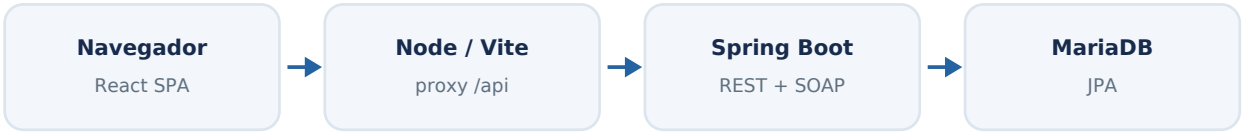
Aplicación web académica para la gestión del gimnasio. Integra interfaz SPA en React, API REST en Spring Boot 4.1.1 y persistencia JPA/Hibernate en MariaDB. Incluye un servicio SOAP de consulta de cursos con WSDL y XSD.

Se sirve mediante Node.js/Vite para desarrollo y se comunica por /api con el backend Java. El proxy permite compartir origen y transmitir la cookie JSESSIONID en desarrollo. En producción debe configurarse servidor web y HTTPS apropiados.

# 2. Arquitectura en capas

| Capa | Responsabilidad | Implementación |
|---|---|---|
| Presentación | Navegación SPA, formularios, vistas, mensajes, validación HTML5/JS y rutas protegidas. | React + Vite + React Router + CSS externos |
| API / Controladores | Recibe HTTP, valida JSON, gestiona sesión y aplica controles por roles. | Spring REST Controllers, HttpSession, @Valid |
| Negocio | Reglas de inscripciones, cancelaciones, pagos y alta transaccional. | @Service, @Transactional, Observer, Singleton, Iterator |
| Persistencia | Consulta y escritura SQL parametrizada; relaciones JPA. | Spring Data JpaRepository + Hibernate |
| Datos | Clientes, usuarios, cursos, máquinas, rutinas, historial, inscripciones, pagos, bajas, consultas y detalles de cursos. | MariaDB / MySQL — 12 tablas |

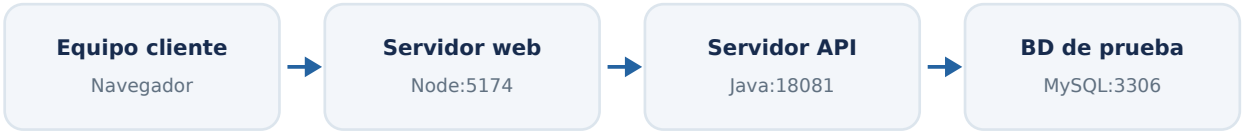
# 3. Diagrama de componentes




| Cliente / navegador | Servidor SPA (Node.js/Vite) | Backend Java | Persistencia |
|---|---|---|---|
| React Router, vistas y formularios | Proxy HTTP /api | Controladores → servicios → repositorios | MariaDB: gimnasio_entregable6_test |

Flujo: Navegador → Vite (5174) → proxy /api → Spring Boot (18081) → JpaRepository → MariaDB (3306). Respuesta JSON hacia React. El endpoint SOAP /ws/cursos publica contrato WSDL/XSD desde Spring Boot.

# 4. Diagrama de despliegue




| Nodo | Servicios / puertos de prueba | Observaciones |
|---|---|---|
| PC Windows 11 | Node.js 22 / Vite 5174; Java 17 / Spring 18081 | Desarrollo y demostración local aislada |
| Base de datos local | XAMPP MariaDB 3306; esquema gimnasio_entregable6_test | Base original gimnasio no utilizada por las pruebas E6 |
| Navegador | http://127.0.0.1:5174 | Usar host 127.0.0.1 de forma coherente |

# 5. Modelo de datos

El esquema original contiene: cliente, usuario, instructores, cursos, maquinas, rutinas e historial_curso. La ampliación agrega: inscripciones, pagos, consultas_contacto, bajas_gimnasio y curso_detalle. La estructura se prepara mediante el script original y la migración idempotente; se prohíbe habilitar ddl-auto=update sobre una base no respaldada.

| Relación | Regla de integridad |
|---|---|
| inscripciones.cliente_id → cliente.cedula | La inscripción pertenece a un cliente. Una inscripción activa duplicada debe rechazarse con HTTP 409. |
| pagos.cliente_id → cliente.cedula | Una deuda vencida pendiente bloquea cancelación y baja. |
| bajas_gimnasio.cliente_id → cliente.cedula | Registro único de baja: impide nuevas inscripciones tras el retiro. |
| curso_detalle.curso_id → cursos.id_curso | Datos de horario, cupos e imagen asociados al curso. |
| historial_curso → cliente, instructores, cursos | No editar claves foráneas al modificar historial. |

# 6. Patrones obligatorios

| Patrón | Uso comprobable |
|---|---|
| MVC | Controladores REST → servicios/repositories y modelos JPA; las vistas son componentes React/JSON. |
| Singleton | CursoService anotado con @Scope("singleton"); otras clases @Service tienen ciclo de vida administrado por Spring. |
| Observer | CursoRegistradoEvent + CursoRegistradoObserver cuando se registra un curso. |
| Iterator | Uso de java.util.Iterator en el recorrido de máquinas y rutinas en servicios; listas iteradas en React. |

# 7. Casos de uso principales

| ID y actor | Secuencia | Resultado esperado |
|---|---|---|
| CU-01 Visitante | Abre inicio, cursos, máquinas, instructores y blog. | Navega sin inicio de sesión. |
| CU-02 Cliente nuevo | Completa registro de perfil y credenciales. | Cuenta y cliente guardados atómicamente; HTTP 201. |
| CU-03 Cliente | Inicia sesión; solicita curso o rutina asignada. | Inscripción persistida; duplicado → HTTP 409. |
| CU-04 Cliente | Consulta inscripciones y pagos; solicita cancelación. | Deuda vencida → HTTP 409; al día → cancelación. |
| CU-05 Cliente | Solicita retiro del gimnasio. | Baja registrada, inscripciones activas canceladas y futuras solicitudes bloqueadas. |
| CU-06 Instructor | Gestiona cursos, máquinas, rutinas e historial. | CRUD condicionado por roles y validaciones. |
| CU-07 Administrador | Gestiona usuarios, cobros, consultas y parámetros. | Solo ADMIN; validaciones y auditoría HTTP. |
| CU-08 Visitante | Envía un mensaje desde contacto. | Consulta persistida y visible al administrador. |

# 8. Contratos REST y SOAP

| Recurso / método | Finalidad / acceso |
|---|---|
| POST /api/usuarios/login; GET /api/usuarios/sesion; POST /api/usuarios/logout | Sesión HttpSession con cookie; roles ADMIN, INSTRUCTOR, CLIENTE. |
| POST /api/registro-clientes | Alta de cliente con cuenta y perfil en transacción. |
| GET/POST /api/inscripciones; GET /api/inscripciones/mias | Registro y consulta de inscripciones. |
| POST /api/inscripciones/{id}/cancelar; POST /api/inscripciones/baja-gimnasio | Cancelación y retiro sujetos a pagos. |
| GET/POST /api/pagos; PUT /api/pagos/{id}/pagar | Consulta y administración de pagos. |
| GET/POST /api/contactos; PUT /api/contactos/{id}/atender | Consultas persistentes y gestión de atención. |
| GET/POST/PUT/DELETE /api/cursos, /api/maquinas, /api/rutinas | Operaciones CRUD y controles por rol. |
| /api/historial-cursos; /api/cursos-detalle; /api/admin/usuarios | Historial, disponibilidad y administración. |
| POST /ws/cursos; GET /ws/cursos.wsdl; GET /ws/cursosSchema.xsd | Consulta SOAP documentada con contratos XML. |

# 9. Seguridad y gestión de errores

- Contraseñas protegidas por BCrypt en usuario; no se persisten contraseñas en localStorage ni sessionStorage.
- La sesión se identifica mediante cookie JSESSIONID; el estado de autorización se comprueba también en el backend.
- La capa de errores usa @RestControllerAdvice y responde con mensajes entendibles; no mostrar stack traces.
- Cabeceras X-Frame-Options: DENY y X-Content-Type-Options: nosniff validadas en prueba HTTP; HSTS solo debe activarse sobre HTTPS real.
- Consultas parametrizadas y JPA reducen riesgo de inyección SQL; utilizar DTOs y Bean Validation.
- El archivo application.properties conserva spring.jpa.hibernate.ddl-auto=none para no mutar estructuras automáticamente.

# 10. Instalación desde cero

1) Clonar repositorio y usar main con los PR finales fusionados, o la rama feature/entregable6-integracion-final para revisión.

2) Instalar Java 17, Maven, Node.js 22, npm, MariaDB (XAMPP) y arrancar MySQL en puerto 3306.

3) PowerShell: .\scripts\preparar-entregable6.ps1 -NombreBase gimnasio_entregable6_test

4) Backend: cd gimnasio-backend\gimnasio-backend ; mvn -B -ntp clean verify

5) Backend: java -jar target\gimnasio-backend-0.0.1-SNAPSHOT.jar --server.port=18081 --spring.datasource.url=jdbc:mysql://127.0.0.1:3306/gimnasio_entregable6_test --spring.jpa.hibernate.ddl-auto=none

6) Frontend: cd gimnasio-frontend ; npm ci ; establecer E6_PROXY_TARGET=http://127.0.0.1:18081 ; npm run dev -- --host 127.0.0.1 --port 5174 --strictPort

7) Navegar a http://127.0.0.1:5174. Probar /api/health, inicio de sesión, consultas, mantenimiento y rutas privadas.

No ejecutar operaciones de escritura sobre la base gimnasio original. El instalador de ampliación incluye comprobaciones de base permitida y SQL idempotente; realizar respaldo antes de usar datos reales.

# 11. Estrategia y resultados de pruebas

| Evidencia de sesión de prueba | Resultado recibido |
|---|---|
| npm run lint | 0 errores, 0 advertencias; 44 archivos. |
| npm run build | Compilación Vite 8.2.2 aprobada. |
| mvn -B -ntp clean verify | BUILD SUCCESS; 9 pruebas Java aprobadas (1 contexto, 2 cursos, 4 inscripciones, 2 SOAP). |
| MariaDB de prueba | 12 tablas después de migración. |
| HTTP sin sesión, roles y cabeceras | Códigos 401, 403, 200 y cabeceras correctas. |
| Inscripciones, mora, cancelación, baja | HTTP 201, 409 y 200 según casos; caso duplicado y bloqueo posterior a baja aprobados. |
| CRUD curso y contacto | Creación, consulta, actualización y eliminación del curso; contacto persistente. |

## 12. Estado y limitaciones verificables

Las pruebas HTTP del 5 de noviembre de 2026 se ejecutaron sobre gimnasio_entregable6_test, no sobre la base original. Falta la revisión visual de todas las pantallas E6 en navegador, prueba de todos los CRUD de interfaz, validación de fotografías específicas y verificación de perfiles de redes sociales; esos puntos deben cerrarse antes de declarar cumplimiento íntegro.

Fuente del alcance: ProyectoP4202620(1).docx (enunciado de Programación 4). Fuente de implementación: PR #17 y PR #18 fusionados y paquete local E6 aplicado en rama de trabajo. Fuente de resultados: consola de la ejecución E6_INTEGRACION_REAL=PASS del 05/11/2026.

Repositorio: https://github.com/marjorm/gimnasio-web-grupoN2